

Homework 13: A Rules Engine

Build eligibility rules out of composed predicates, use all four method reference forms, then walk into the capture rule on purpose.

The Data

```
record Applicant(String name, int age, int creditScore, boolean employed) { }

List.of(
    new Applicant("ada",      36, 780, true),
    new Applicant("grace",    45, 610, true),
    new Applicant("alan",     17, 800, false),
    new Applicant("edsger",   52, 550, true),
    new Applicant("barbara",  29, 720, false));
```

Part One: Composed Predicates

Create `RulesEngine.java` in the classic form.

Write **three separate named predicates**:

Name	Rule
<code>isAdult</code>	age 18 or over
<code>hasGoodCredit</code>	credit score 700 or over
<code>isEmployed</code>	employed is true

Then build two composite rules **by combining them**, not by writing a new lambda:

- `eligible` — adult, good credit, and employed
- `needsReview` — adult, but credit below 700

Print a row per applicant showing all three results plus whether they are a minor, using `negate()` for the last one.

Write a comment on why three named predicates beat one large lambda. Your answer should mention reuse: one of the two composite rules shares a clause with the other.

Part Two: All Four Method Reference Forms

Use each form at least once, each with a comment naming which it is:

1. A **static** method reference

2. A **constructor** reference
3. A **bound instance** reference
4. An **unbound instance** reference

Then Explain The Hard Part

Forms 3 and 4 are both written `Something::method`. Write a comment explaining, in your own words:

- What sits to the left of the `::` in each case, and why that is the tell
- Which one picks its receiver once, and which one picks it per call
- **The observable consequence for arity.** A bound reference to a one-argument method needs a functional interface with how many parameters? What about the unbound reference to the same method? Show both.

Finally, use `andThen` to build a pipeline turning an `Applicant` into a credit band of A, B or C, and print it for everyone.

Part Three: The Capture Rule

First, write the version that does **not** compile: a local `int approved = 0;` incremented inside a `forEach`. Confirm the error, then put it in a comment along with an explanation of why the rule exists.

Then write the same count **three** ways that do work:

1. The `int[] box = {0}` trick
2. An ordinary `for` loop
3. A stream, using `filter` and `count`

The Question

Write a comment explaining why the array trick compiles when the plain `int` did not, and why it is still a bad answer. Your explanation must distinguish between the **variable** and the **object it points at**, because that distinction is the whole rule.

Finish by capturing a `List` in a lambda and mutating it, showing that this is legal, then say why `.toList()` is better anyway.

Acceptance Criteria

- ☐ Runs with `java RulesEngine.java`
- ☐ Three separately named predicates exist
- ☐ `eligible` and `needsReview` are built by composing them, not rewritten
- ☐ `negate()` is used

- ☐ A comment justifies composition, mentioning the shared clause
- ☐ All four method reference forms appear, each labelled
- ☐ The bound-versus-unbound explanation covers the arity consequence and shows both signatures
- ☐ An `andThen` pipeline produces credit bands
- ☐ The non-compiling capture attempt is quoted with its real error
- ☐ The count is produced three working ways, all agreeing
- ☐ The explanation distinguishes the variable from the object

Stretch

Write a `Predicate<Applicant>` factory: a method taking a minimum score and returning a predicate that tests against it.

Then write a comment on what that method captures, whether the captured parameter is effectively final, and why a method parameter is allowed to be captured at all given everything this module said about stack frames.

Hint, if `compose` will not type-check

`a.andThen(b)` runs `a` first. `a.compose(b)` runs `b` first. Only one ordering lines up when `a` takes an `Applicant` and `b` takes an `int`. If the compiler refuses, you have them the wrong way round, and that refusal is doing you a favour.